

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot.

8. Experimental Results: Plots and Interpretation

8.1 Feature Histograms

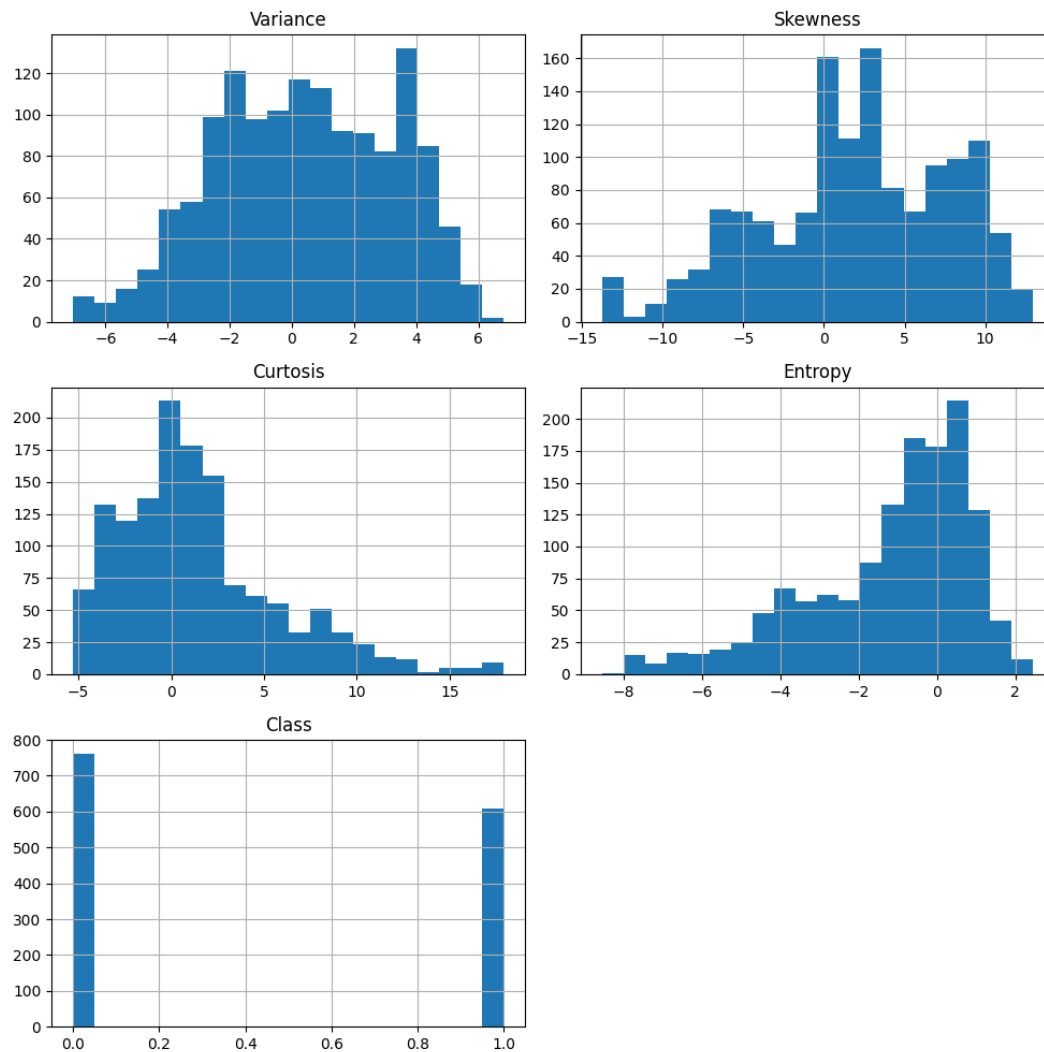


Figure 1: Feature Histograms

Interpretation:

Variance: It ranges from -7 to 6 in the graph. The variance is moderately spread. It is not high in contrast (most of the samples point to one-to-two variances). It is also not very smooth either. (all the variances in x axis have same value on the y axis).

Skewness: The graph shows that it is right skewed (Asymmetrical). Skewness tells us about the asymmetry.

Kurtosis: From the graph, we can infer that there are plenty number of outliers as there is a peak.

Entropy: It is zero centered, so we can say that there is moderate variability.

Class: From this histogram, we can say that class 0 (genuine notes) have comparably more samples than class 1 (fake notes). This is a balanced data set with almost similar number of

samples for each class.

8.2 Correlation Heatmap

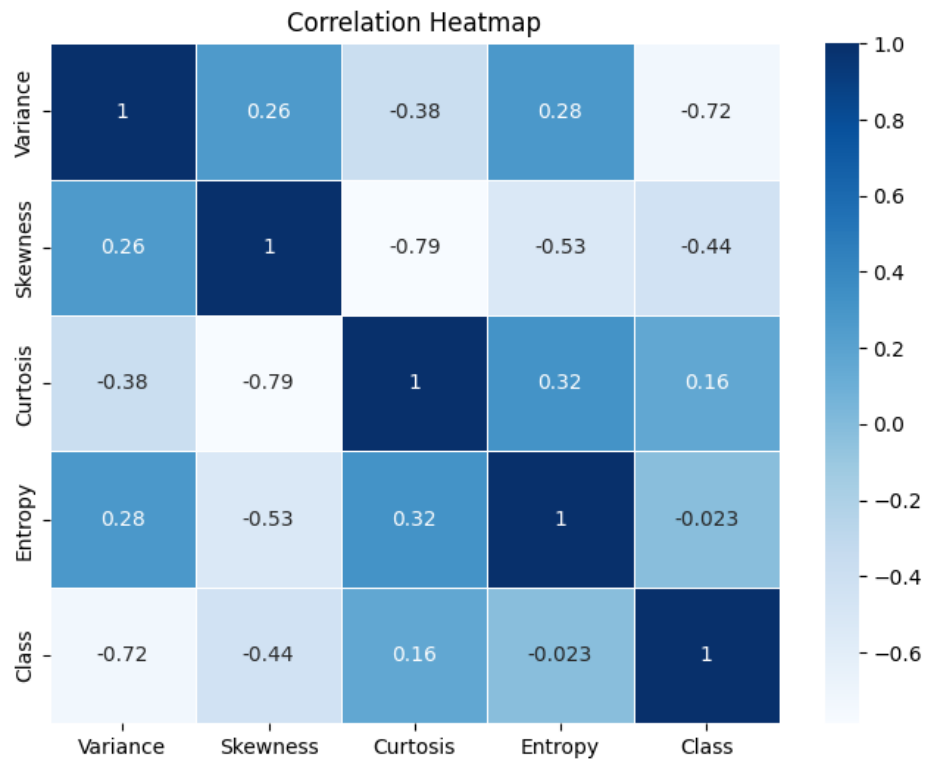


Figure 2: Correlation Heatmap

Interpretation:

From this map, we can say that skewness and kurtosis are highly negatively correlated (as skewness increases kurtosis drastically decreases).

8.3 Scatter Plot

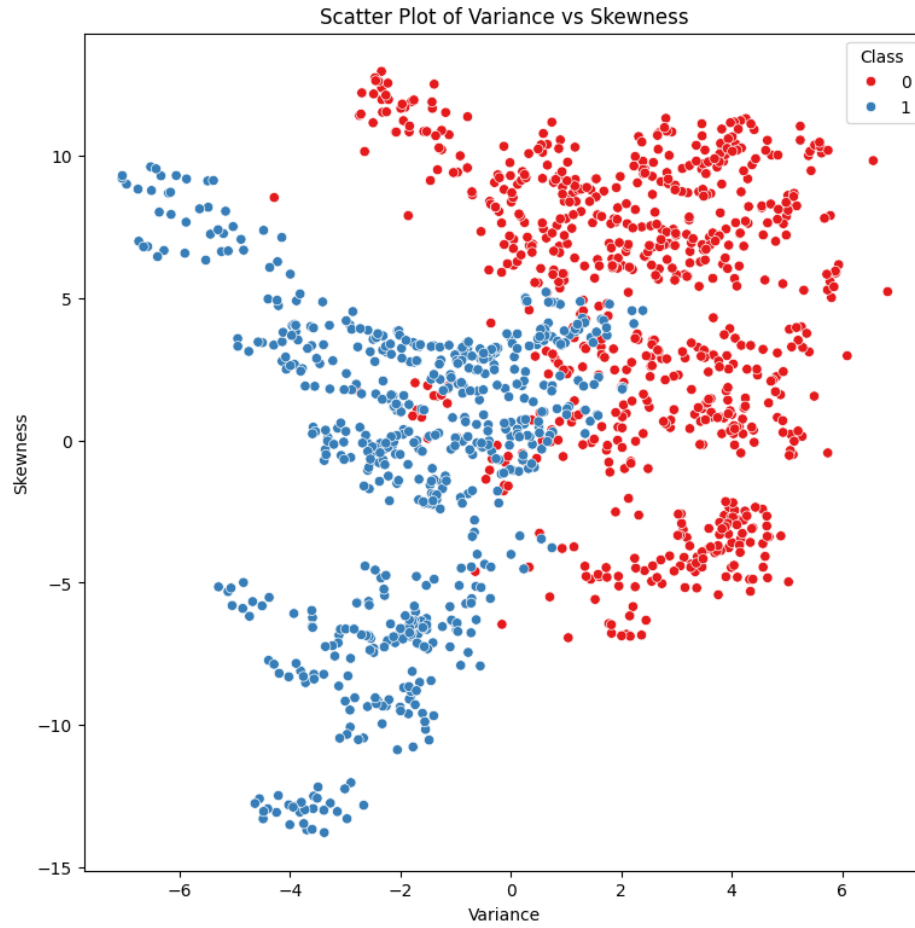


Figure 3: Scatter Plot

Interpretation:

The points of class 1 are towards left region and the points of the class 0 are towards right region. so , we can say that these datapoints can be classified linearly.

8.4 Training Error vs Epoch



Figure 4: Training Error vs Epoch

Interpretation:

As the number of training epochs increases, the error decreases as the model explores the data.

8.5 Weight Evolution

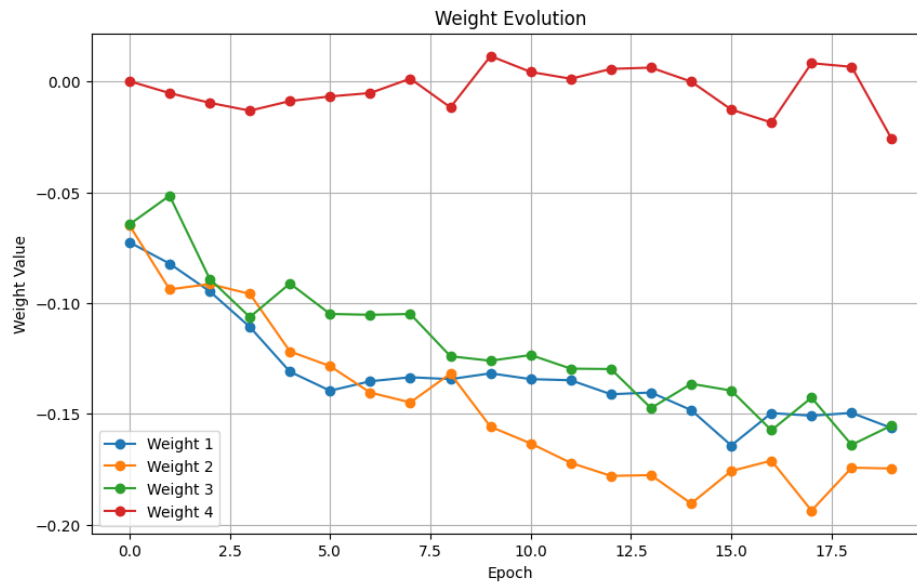


Figure 5: Weight Evolution

Interpretation:

From this plot, weights 1,2,3 needed larger changes while weight 4 changed slightly. This means weight 4 does not contribute much in minimizing error.

8.6 Bias Evolution

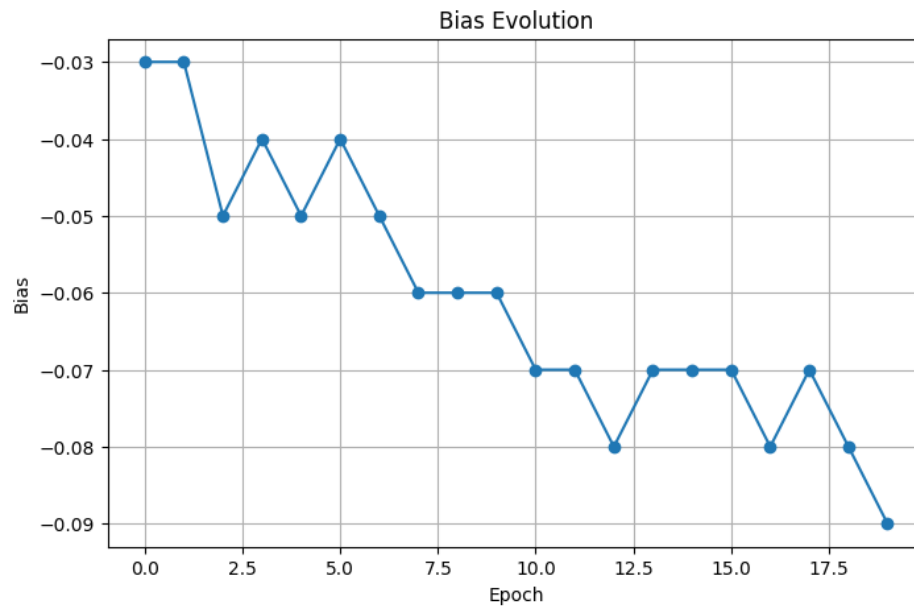


Figure 6: Bias Evolution

Interpretation:

This graph shows that the bias needs to be decreased in large values to reduce the training error with increasing training epochs.

8.7 Confusion Matrix

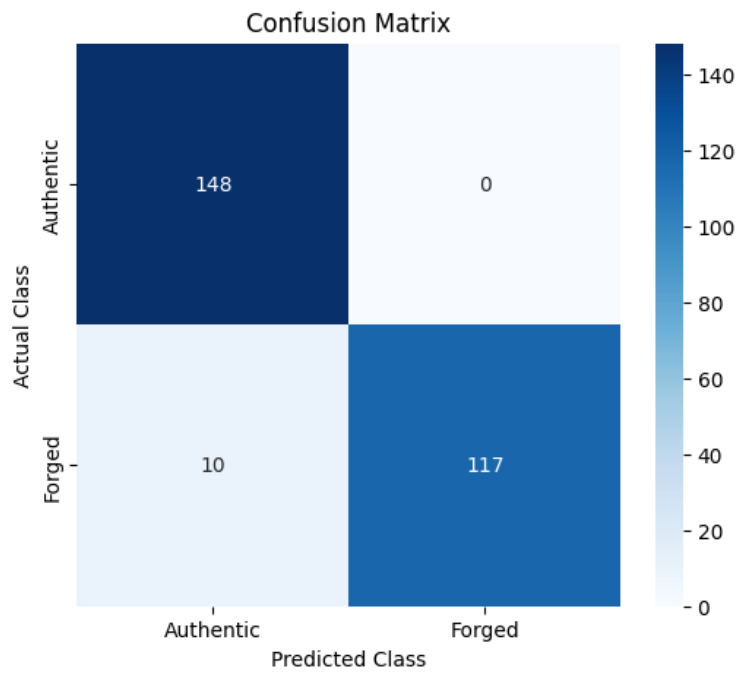


Figure 7: Confusion Matrix

Interpretation:

Here we can see that True positive and True negative values are high . It means that it correctly classified true notes as true and false notes as false.

8.8 Learning Rate Comparison

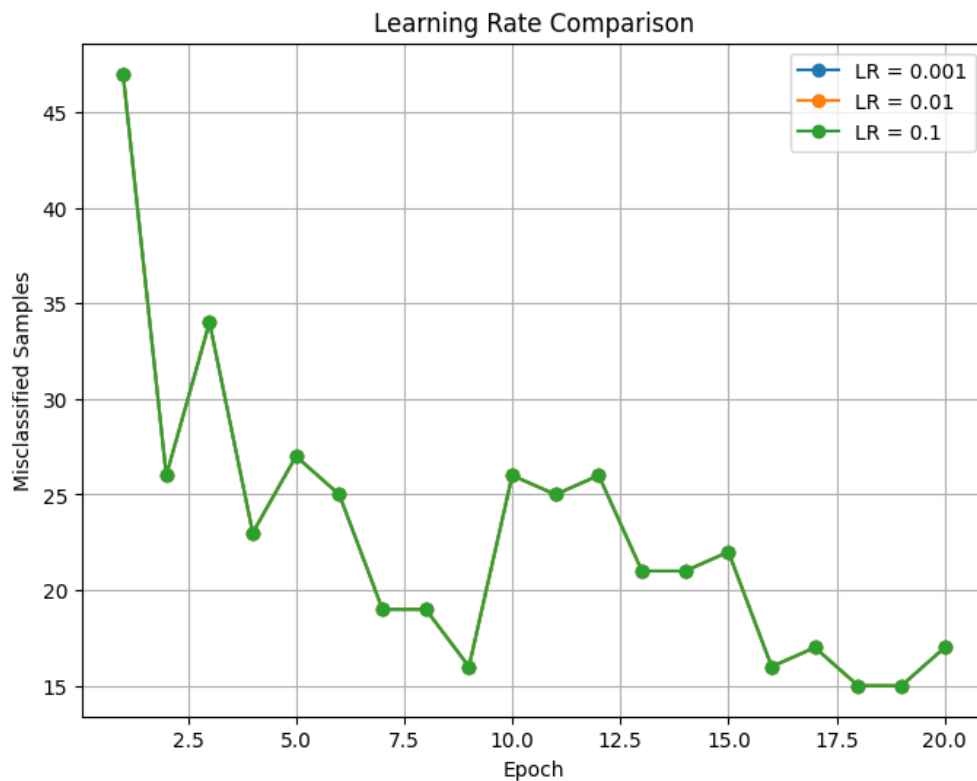


Figure 8: Learning Rate Comparison

Interpretation:

Here the learning rate graphs are identical because the banknote authentication dataset is nearly linearly separable. This means that there is a straight line that classifies the data. Once the perceptron starts moving toward that separating boundary, all three learning rates 0.001, 0.01, and 0.1 found the same solution.

8. Performance Tables

Training Summary

Dataset Size	1372 samples
Train/Test Split	80% Training, 20% Testing (1097 train, 275 test)
Learning Rate	0.1
Epochs	20
Final Weights	Replace with the final weight vector obtained from the program
Final Bias	Replace with the final bias value obtained from the program
Accuracy	96.36%
Precision	100.00%
Recall	92.13%
F1-score	95.90%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	47	-0.0725	-0.0649	-0.03
2	26	-0.0822	-0.0938	-0.03
3	34	-0.0947	-0.0914	-0.05
4	23	-0.1107	-0.0958	-0.04
5	27	-0.1310	-0.1218	-0.05
6	25	-0.1395	-0.1284	-0.04
7	19	-0.1353	-0.1404	-0.05
8	19	-0.1335	-0.1448	-0.06
9	16	-0.1343	-0.1318	-0.06
10	26	-0.1317	-0.1558	-0.06
11	25	-0.1343	-0.1633	-0.07
12	26	-0.1348	-0.1721	-0.07
13	21	-0.1411	-0.1780	-0.08
14	21	-0.1403	-0.1776	-0.07
15	22	-0.1483	-0.1904	-0.07
16	16	-0.1642	-0.1758	-0.07
17	17	-0.1496	-0.1711	-0.08
18	15	-0.1508	-0.1937	-0.07
19	15	-0.1495	-0.1743	-0.08
20	17	-0.1564	-0.1747	-0.09

9. Additional Tasks

1. Comparison of Step and Sigmoid Activation Functions

The step activation function only produces 0 or 1. It is discrete which makes it non differentiable. If the function is not differentiable then gradient descent is not possible for back propagation.

Sigmoid Activation Function gives continuous values from 0 to 1 which makes it differentiable which is helpful in gradient descent for back propagation.

2. Comparison with Scikit-learn's Perceptron

The perceptron we implemented is for understanding the underlying concepts like weight updates, forward pass, backward pass which we cannot see if we use scikit-learn's perceptron. The scikit learn's perceptron enables us to perform perceptrons efficiently , better abstraction , evaluate accurately.

3. Effect of Different Learning Rates

The experiment was repeated using learning rates of 0.001, 0.01, and 0.1.

Learning rate = 0.001: Training converged very slowly, requiring more epochs to reduce the classification error.

Learning rate = 0.01: Produced stable learning with gradual convergence and improved accuracy.

Learning rate = 0.1: Achieved the fastest convergence with the fewest misclassified samples while remaining stable throughout training.

Thus, a learning rate of 0.1 provided the best performance in this experiment.

4. Why a Single Layer Perceptron Cannot Solve XOR

The XOR problem can only be solved using non linear decision boundary as the points on the diagonal line are of same class. Single layer perceptron can only classify linear data . It can only produce linear decision boundary. Multi layer perceptron with at least one hidden layer can be used to classify non linear data. so, MLP is used to solve XOR problem.

5. Effect of Feature Normalization

Feature normalization scales all input features to a similar numerical range. It prevents features with larger values from dominating the learning process. It results in faster convergence. It improves classification accuracy. Without normalization, training becomes slower and may converge to a less optimal solution.

Conclusion

Through this experiment, I learned how a single layer perceptron works and classifies the real world data. I understood the importance of data preprocessing, feature normalization, and weight updates in improving model performance. The model achieved an accuracy of 96.36 percent . This shows that it can differentiate between actual and forged banknotes effectively. This experiment also helped me understand the strengths and limitations of the perceptron algorithm that it cannot classify non linear data.

10. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results

- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

12. Source Code Repository

The source code and supporting files for this experiment are available on GitHub.

GitHub Repository:

[https://github.com/Nitheesha-1025/Deep-Learning/tree/Perceptron-Algorithm-\(-Experiment-1\)](https://github.com/Nitheesha-1025/Deep-Learning/tree/Perceptron-Algorithm-(-Experiment-1))